



Local LLM Inference on Heterogeneous Hardware

Architecture and configuration of the AIOS runtimes: llama.cpp, stable-diffusion.cpp, ggml and Vulkan

Abstract

AIOS is a self-contained x86-64 appliance that turns bare metal or a virtual machine into a local inference server for open-weight language models, with an administration portal, an OpenAI-compatible API and Open WebUI. This paper documents the inference path in detail: how llama.cpp is built as a set of dynamically loaded ggml backends (fourteen x86 CPU variants and a Vulkan backend), how GPUs of every vendor are reached through one Vulkan implementation per driver stack, how the appliance chooses devices, fits the context window to measured memory, starts llama-server with explicit parameters and falls back to the CPU when a GPU cannot load a model, and how multimodal projectors, the reasoning budget and the request gateway are configured. Every parameter quoted is the one the shipped code passes, and every llama.cpp default is taken from the pinned commit 5266f24. Measurements come from the project's lab appliance; their limits are stated.

Table 1. Software baseline of the image described in this paper.

Component	Version	Role
Ubuntu	24.04 LTS (noble)	base system, packages from the Ubuntu archive
Linux kernel	7.0.0-31-generic (HWE)	GPU kernel drivers: i915/xe, amdgpu, nouveau, NVIDIA modules
llama.cpp / ggml	commit 5266f24da75d...	language inference engine, llama-server (port 8090)
stable-diffusion.cpp	commit cc515a01f9d0...	image inference engine, sd-server (port 8091)
Mesa	25.2.8 (LLVM 20.1.2)	Vulkan ICDs: ANV, hasvk, RADV, NVK, lavapipe
Vulkan loader	1.3.275	ICD discovery and dispatch
NVIDIA driver	580.178.04 (server branch)	Vulkan ICD and kernel modules (open and proprietary)
Open WebUI	0.11.3	chat front end, OpenAI client of the gateway
Python / NGINX	3.12.3 / 1.24.0	control plane runtime / TLS reverse proxy



Local LLM Inference on Heterogeneous Hardware

Architecture and configuration of the AIOS runtimes: llama.cpp, stable-diffusion.cpp, ggml and Vulkan

Contents

1. Introduction	4
2. System architecture	4
3. Building llama.cpp	5
3.1 Build configuration	5
3.2 Dynamic backend discovery	6
4. The Vulkan backend	7
4.1 Why Vulkan	7
4.2 Instance and device enumeration	8
4.3 Memory reporting	8
4.4 Buffer allocation	8
4.5 Kernel side: HWE kernel, amdgpu and NVIDIA modules	9
5. Device discovery and selection in AIOS	9
5.1 Probe	9
5.2 Selection policy and device arguments	10
5.3 Start sequence and CPU fallback	10
6. Memory model and context fitting	11
6.1 KV cache estimate	11
6.2 Compatibility rating	11
6.3 Context fitting at start and interaction with --fit	11
7. The llama-server invocation	12
8. Reasoning budget	13
9. Multimodal input	13
10. Request path	14
10.1 NGINX	14
10.2 Gateway semantics	14
10.3 Open WebUI configuration	14
11. Model ingestion and validation	15
12. Isolation of the inference services	15
13. Verification and measurements	16
14. Limitations and future work	16
15. The image engine	17
References	17

1. Introduction

Running a language model locally is a memory problem before it is a compute problem. Autoregressive decoding reads every weight of a dense model for every generated token, so the token rate is bounded by memory bandwidth divided by model size, while prompt processing is bounded by compute. An appliance that must work on whatever hardware it is installed on — a two-core VM, a workstation with an integrated GPU, a server with several discrete cards — therefore has to answer three questions at every model start: “which devices can run tensors”, “how much memory each really has”, and “how large a context the model can be given”. AIOS answers them explicitly in its own control plane and then hands llama.cpp a fully specified command line, instead of relying on implicit defaults.

The design constraints are: no vendor assumption (the same image must use NVIDIA, AMD and Intel GPUs, dedicated or integrated, or none); no network dependency after models are downloaded; no configuration on the operator's side; and honest failure modes — a model that does not fit is refused with a reason, and a GPU that fails never costs the model.

Section 2 describes the system architecture. Sections 3 and 4 cover the llama.cpp build and the Vulkan backend. Section 5 describes device discovery and selection, Section 6 the memory model and context fitting, Section 7 the exact llama-server invocation, Sections 8–9 reasoning budget and multimodal input, Section 10 the request gateway, Section 11 model ingestion, Section 12 service isolation, Section 13 measurements and Section 14 known limitations.

2. System architecture

The appliance is a Ubuntu 24.04 image with no desktop and no container runtime. The raw GPT disk holds a 1 MiB BIOS boot partition, a 256 MiB FAT32 ESP, a 10 GiB ext4 root and an ext4 data partition mounted at /var/lib/aio that grows to the end of the medium at boot. Models, runtime logs and the SQLite database live on the data partition; the root file system carries the binaries, the Vulkan drivers and the bundled NVIDIA module packages.

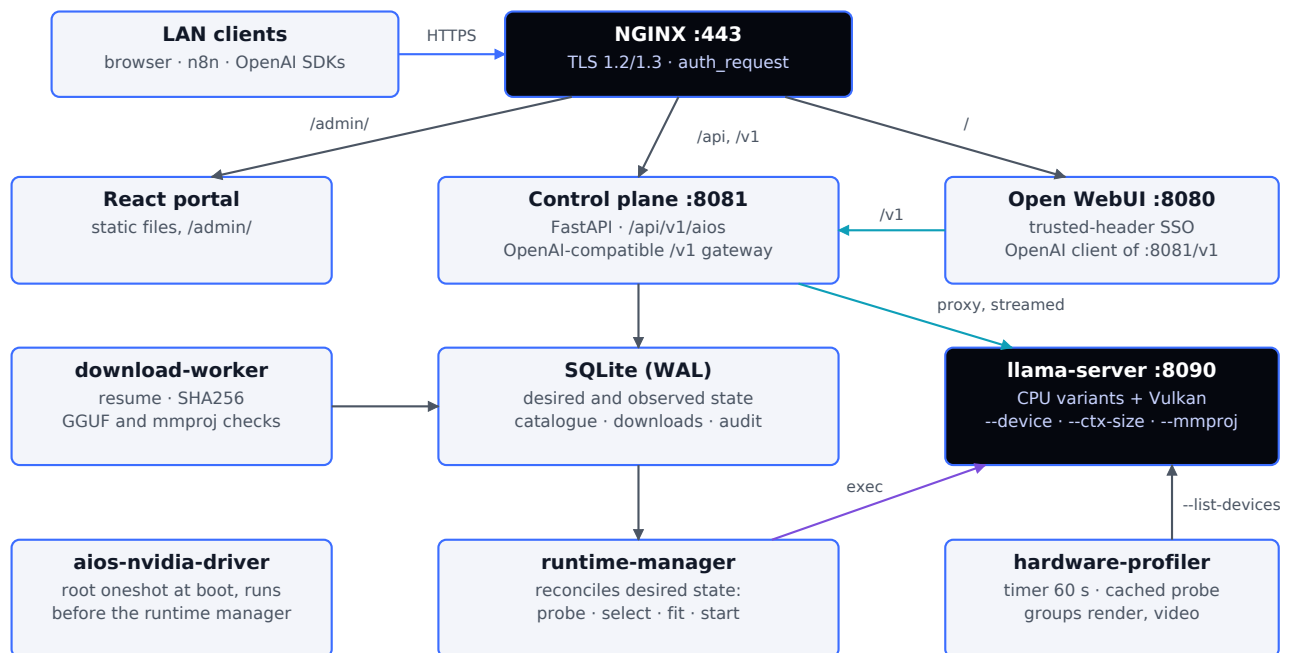


Figure 1. Process architecture. Only NGINX listens on the LAN; every other service is bound to loopback. The control plane records the desired state of each runtime in SQLite; the runtime manager owns the llama-server process.

The separation between the control plane and the runtime manager is deliberate. The HTTP API only writes a desired state (RUNNING, STOPPED, RESTART) into `runtime_instances`; `aios-runtime-manager` reconciles it once per second, spawns and supervises `llama-server`, and writes back the observed state (STARTING, RUNNING, FAILED). A model that takes minutes to load, or crashes, never blocks the API. Version 1 keeps one model in memory; the schema already models one instance per model.

Table 2. Services involved in inference.

Unit	Type / user	Responsibility
<code>aios-control-plane</code>	simple / aios	REST API, sessions, catalogue, OpenAI-compatible gateway on 127.0.0.1:8081
<code>aios-runtime-manager</code>	simple / aios + render, video	desired-state loop, GPU probe, context fit, llama-server lifecycle, CPU fallback
<code>aios-download-worker</code>	simple / aios	persistent download queue, resume, SHA256, GGUF validation, projector download
<code>aios-hardware-profiler</code>	oneshot + 60 s timer / aios + render, video	hardware inventory, cached GPU probe (refreshed every 10 min or after boot)
<code>aios-nvidia-driver</code>	oneshot / root	installs and loads the NVIDIA kernel module flavour matching the cards present
<code>aios-repository-sync</code>	oneshot + timer / aios	metadata-only discovery from Hugging Face, ModelScope, GitHub releases or manifests
<code>aios-open-webui</code>	simple / aios-webui	chat UI; its OpenAI connection points at the gateway with the local inference key
<code>aios-platform</code>	simple / root	closed set of validated privileged operations (network, TLS, power, backup, updates)

3. Building llama.cpp

3.1 Build configuration

`llama.cpp` is compiled inside the Ubuntu build root from a pinned source archive by `scripts/build-runtime.sh`. The configuration makes every compute backend a shared library selected at run time, so one binary serves every CPU generation and every GPU vendor:

```
cmake -S llama.cpp-5266f24... -B /usr/src/llama-build -G Ninja -DCMAKE_BUILD_TYPE=Release \
  -DGGML_NATIVE=OFF -DGGML_BACKEND_DL=ON -DGGML_CPU_ALL_VARIANTS=ON -DGGML_VULKAN=ON \
  -DLLAMA_CURL=ON -DLLAMA_BUILD_TESTS=OFF -DLLAMA_BUILD_EXAMPLES=ON -DLLAMA_BUILD_SERVER=ON \
  -DCMAKE_INSTALL_PREFIX=/opt/aios/runtime -DCMAKE_INSTALL_RPATH=/opt/aios/runtime/lib
cmake --build /usr/src/llama-build --target llama-server ggml-vulkan -j2 \
  || cmake --build /usr/src/llama-build --target llama-server ggml-vulkan -j1
```

Table 3. CMake options and their effect.

Option	Effect
<code>GGML_NATIVE=OFF</code>	no <code>-march=native</code> : the build host's CPU must not leak into the binary
<code>GGML_BACKEND_DL=ON</code>	backends are <code>libggml-<name>[-variant].so</code> modules loaded with <code>dlopen</code> by <code>ggml_backend_load_all()</code>
<code>GGML_CPU_ALL_VARIANTS=ON</code>	one CPU backend per instruction-set level (Table 4); each exports <code>ggml_backend_score()</code>
<code>GGML_VULKAN=ON</code>	builds <code>libggml-vulkan.so</code> ; 152 GLSL compute shaders are compiled to SPIR-V with <code>glslc</code> and embedded (the build defines <code>GGML_VULKAN_COOPMAT_GLSLC_SUPPORT</code>)
<code>LLAMA_CURL=ON</code>	links <code>libcurl</code> (not used by AIOS, which downloads through its own verified worker)
<code>-j2</code> , then <code>-j1</code>	several generated shader translation units need multiple GiB each to compile; a failed parallel build resumes serially in the same build directory

The build directory is stamped with "`<commit> cpu-all vulkan`"; an interrupted build of the same commit and options resumes instead of starting over. The build-time packages `glslc`,

libvulkan-dev and spirv-headers are purged from the final image together with the compilers.

3.2 Dynamic backend discovery

With GGML_BACKEND_DL, ggml_backend_load_best(name) enumerates files named libggml-<name>-.so in its search paths, dlopens each one, calls its ggml_backend_score() and keeps the highest non-zero score; if none scores it tries the plain libggml-<name>.so. The search paths are, in order, GGML_BACKEND_DIR (a compile-time definition, not set in this build), the directory of the executable (resolved through /proc/self/exe) and the current working directory. The shared libraries themselves are installed in /opt/aios/runtime/lib and the executable in /opt/aios/runtime/bin, so the image links every backend into bin/:

```
for backend in /opt/aios/runtime/lib/libggml-cpu*.so /opt/aios/runtime/lib/libggml-vulkan.so; do
  ln -sf "$backend" "/opt/aios/runtime/bin/$(basename "$backend")"
done
```

A backend absent from the search paths is not an error for ggml — it is simply never offered — which makes this step easy to get wrong silently. An early build of this image linked only the CPU variants, and every GPU remained unused without any log line. The image build now asserts the result on the finished root file system: with /proc mounted in the chroot, GGML_VK_VISIBLE_DEVICES=0 llama-server --list-devices must report Vulkan0 (Mesa's software renderer stands in for a GPU the build host does not have), or the build fails.

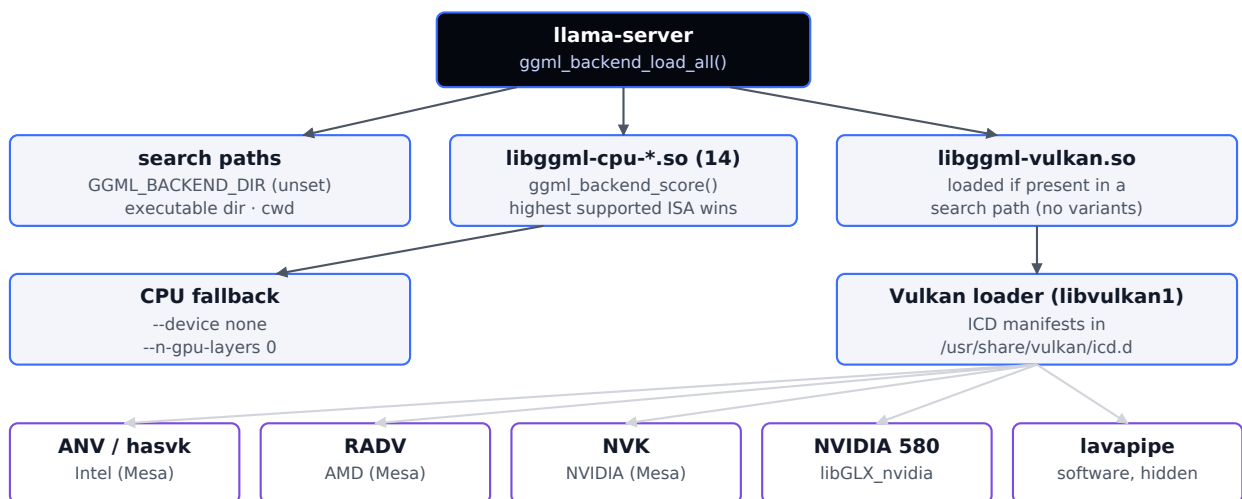


Figure 2. Backend resolution at run time. The CPU backend with the best score for the running processor is chosen; the Vulkan backend exposes every physical device the Vulkan loader enumerates through the installed ICDs.

Table 4. CPU backend variants built for x86-64 (from ggml/src/CMakeLists.txt at the pinned commit).

Variant	Required instruction sets
x64	baseline x86-64
sse42	SSE4.2
sandybridge	SSE4.2 AVX
ivybridge	SSE4.2 AVX F16C
piledriver	SSE4.2 AVX F16C FMA
haswell	SSE4.2 AVX F16C FMA AVX2 BMI2
skylakex	... AVX2 BMI2 AVX512
cannonlake	... AVX512 AVX512_VBMI
cascadelake	... AVX512 AVX512_VNNI
icelake	... AVX512 AVX512_VBMI AVX512_VNNI
cooperlake	... AVX512 AVX512_VNNI AVX512_BF16
zen4	... AVX512 AVX512_VBMI AVX512_VNNI AVX512_BF16
alderlake	... AVX2 BMI2 AVX_VNNI
sapphirerapids	... AVX512_VBMI AVX512_VNNI AVX512_BF16 AMX_TILE AMX_INT8

4. The Vulkan backend

4.1 Why Vulkan

CUDA reaches NVIDIA only, ROCm a subset of AMD GPUs, SYCL/oneAPI Intel; each adds gigabytes of runtime and its own supported-hardware matrix. Vulkan compute is implemented by every Linux GPU driver stack that matters here, so one ggml backend covers discrete and integrated GPUs of all three vendors, including GPUs passed through to a virtual machine. The cost is performance on NVIDIA hardware, where CUDA kernels can be faster for some operations; the benefit is a single image whose GPU support is a property of the drivers it carries, not of the build.

Table 5. Vulkan implementations shipped in the image.

Hardware	Vulkan ICD	Kernel driver	Notes
Intel Gen9+ (Skylake and newer iGPU), Arc A/B series	Mesa ANV	i915 / xe	Battlemage, Lunar Lake and Panther Lake need the 7.0 HWE kernel
Intel Gen7-8 (Ivy Bridge-Broadwell iGPU)	Mesa hasvk	i915	limited feature set
AMD Radeon HD 7000 and newer, APUs	Mesa RADV	amdgpu	SI/CIK moved from radeon to amdgpu via modprobe options
NVIDIA Maxwell and newer	NVIDIA 580 (libGLX_nvidia)	nvidia (open or proprietary)	flavour chosen per card at boot (\$4.5)
NVIDIA Kepler and older	Mesa NVK (if supported)	nouveau	fallback when the nvidia module refuses the card
none (CPU)	Mesa lavapipe	—	never offered as a GPU unless AIOS_GPU_ALLOW_SOFTWARE=1

4.2 Instance and device enumeration

`ggml_vk_instance_init()` requires a Vulkan 1.2 instance. Without `GGML_VK_VISIBLE_DEVICES`, it walks the physical devices and keeps those whose type is `DISCRETE_GPU` or `INTEGRATED_GPU` and that pass `ggml_vk_device_is_supported()` — which at this commit requires the Vulkan 1.1 feature `storageBuffer16BitAccess`. Devices exposed twice by different drivers (for example `RADV` and `AMDVLK`) are de-duplicated by device UUID. CPU-type devices such as `llvmpipe` are skipped by default; setting `GGML_VK_VISIBLE_DEVICES` replaces the whole policy with an explicit list of indices, which is how the lab exercises the GPU path without a GPU.

4.3 Memory reporting

`ggml_backend_vk_get_device_memory()` sums heap sizes: for a discrete GPU only `DEVICE_LOCAL` heaps, for an integrated GPU every heap. When `VK_EXT_memory_budget` is available, free memory is `heapBudget - heapUsage` per heap; otherwise the full heap size is reported as free. These are the figures printed by `llama-server --list-devices` as `VulkanN: NAME (TOTAL MiB, FREE MiB free)`, which AIOS parses (§5). For an integrated GPU the reported heaps are carved out of system RAM, so they must never be added to it.

4.4 Buffer allocation

Device buffers are allocated with an ordered list of acceptable memory property sets. On UMA devices the backend first asks for `DEVICE_LOCAL | HOST_VISIBLE | HOST_COHERENT` memory, so tensors can be read and written through a mapping, then falls back to device-local and finally host-visible memory. Allocations are capped by `maxMemoryAllocationSize` (and `maxBufferSize` when reported) and sub-allocated in blocks of 1 GiB by default. Weights offloaded to an integrated GPU are copied into such buffers: they occupy system memory in addition to the page-cache copy of the memory-mapped file, which matters for large models (§14).

Table 6. `ggml-vulkan` environment variables most relevant to operators (read at the pinned commit).

Variable	Effect
<code>GGML_VK_VISIBLE_DEVICES</code>	explicit comma-separated device indices; bypasses the discrete/integrated filter
<code>GGML_VK_PREFER_HOST_MEMORY</code>	allocate device buffers in host-visible memory first
<code>GGML_VK_DISABLE_HOST_VISIBLE_VIDMEM</code> / <code>GGML_VK_ALLOW_SYSTEM_FALBACK</code>	restrict discrete-GPU buffers to device-local memory / allow falling back to system memory
<code>GGML_VK_FORCE_MAX_ALLOCATION_SIZE</code> / <code>GGML_VK_SUBALLOCATION_BLOCK_SIZE</code>	override allocation limits (bytes); useful for drivers that under-report
<code>GGML_VK_DISABLE_F16</code> / <code>_BFLOAT16</code> / <code>_INTEGER_DOT_PRODUCT</code>	disable shader feature paths on drivers with defects
<code>GGML_VK_DISABLE_COOPMAT</code> / <code>_COOPMAT2</code>	disable cooperative-matrix shaders (<code>VK_KHR_cooperative_matrix</code> / <code>NV coopmat2</code>)
<code>GGML_VK_DISABLE_FUSION</code> / <code>_GRAPH_OPTIMIZE</code> / <code>_MMVQ</code> , <code>GGML_VK_FORCE_MMVQ</code>	toggle operator fusion, graph optimisation and quantised matmul-vector kernels
<code>GGML_VK_PERF_LOGGER</code> / <code>GGML_VK_MEMORY_LOGGER</code>	per-operation timing and allocation logging for diagnosis

AIOS sets none of these in production. They are listed because they are the supported way to work around a driver defect on a specific machine without rebuilding the image (a `systemd` drop-in for `aios-runtime-manager`).

4.5 Kernel side: HWE kernel, amdgpu and NVIDIA modules

The Ubuntu 24.04 GA kernel (6.8) predates Intel Battlemage, Lunar Lake and Panther Lake graphics and AMD Radeon RX 9000; on it those GPUs expose no DRM device and therefore no Vulkan device. The image carries the hardware enablement kernel (linux-image-generic-hwe-24.04, 7.0.0-31) and removes the GA kernel. Firmware for those GPUs is present in the 24.04 linux-firmware packages. Radeon HD 7000 to R9 200 (SI/CIK) are bound to amdgpu instead of the legacy radeon module, which has no Vulkan driver:

```
# /etc/modprobe.d/aio-amdgpu.conf
options radeon si_support=0 cik_support=0
options amdgpu si_support=1 cik_support=1
```

NVIDIA publishes two kernel module flavours and neither covers all cards: the open modules are mandatory from the RTX 50 series and support Turing and newer; Pascal and Maxwell require the proprietary modules. The image carries the prebuilt Ubuntu packages for both flavours of the 580-server branch, matched to its kernel, under /opt/aio/drivers/nvidia/{open,closed}. At boot, aio-nvidia-driver (root, oneshot, Before=aio-runtime-manager) decides from PCI data alone:

```
ids = PCI device ids with vendor 0x10de and class 0x03xxxx (display controllers)
no ids -> exit
any id < 0x1e00 -> flavour = closed # a pre-Turing card: proprietary modules drive both generations
otherwise -> flavour = open
dpkg --purge <other flavour>
dpkg -i /opt/aio/drivers/nvidia/<flavour>/*-$(uname -r)*.deb # offline
no bundled debs for this kernel -> apt-get install the matching linux-modules-nvidia package
depmod -a; modprobe nvidia && modprobe nvidia-uvdm
udevadm trigger --subsystem-match=pci --action=add # creates /dev/nvidia*
modprobe nvidia fails -> modprobe nouveau # Kepler and older: try Mesa NVK
state -> /var/lib/aio/system/nvidia-driver.json {flavour, kernel, status, detail}
```

The proprietary flavour links nvidia.ko from object files at install time, which is why binutils is kept in the image. The recorded state is shown on the portal's Hardware page, so a driver that failed to load is distinguishable from a machine without a GPU.

5. Device discovery and selection in AIOS

5.1 Probe

accelerators.detect() takes the device list from llama.cpp itself, because only what the bundled engine can open is useful, and enriches it with the device type from vulkaninfo --summary:

```
$ LD_LIBRARY_PATH=/opt/aio/runtime/lib llama-server --list-devices # timeout 60 s
Available devices:
  Vulkan0: NVIDIA GeForce RTX 3090 (24576 MiB, 23800 MiB free)
  Vulkan1: Intel(R) Iris(R) Xe Graphics (ADL GT2) (15872 MiB, 12000 MiB free)
$ vulkaninfo --summary # timeout 30 s
GPU0: deviceType = PHYSICAL_DEVICE_TYPE_INTEGRATED_GPU   deviceName = Intel(R) Iris(R) Xe Graphics (ADL GT2)
GPU1: deviceType = PHYSICAL_DEVICE_TYPE_DISCRETE_GPU      deviceName = NVIDIA GeForce RTX 3090
```

Lines are parsed with `^\s*(\w+\d+):\s*(.+?)\s*((\d+) MiB,\s*(\d+) MiB free\)`. Each llama.cpp device is matched to a physical device by name containment; the Vulkan type maps to discrete, integrated, virtual or software, and an unmatched device is unknown, which is treated like a discrete GPU so that a driver reporting names differently does not lose its GPU. Software devices are dropped unless AIOS_GPU_ALLOW_SOFTWARE=1, in which case they are typed integrated (they render in system RAM, as an iGPU would). The result is cached in system/accelerators.json.

Probing starts llama.cpp and initialises every GPU driver, so it happens only in processes that may open GPU device nodes: the runtime manager probes afresh before every start, and the hardware profiler reuses a probe younger than 600 s taken during the current boot (a cache written before `psutil.boot_time()` is ignored, since a GPU or its driver may have appeared since). The control plane has no access to `/dev/dri` and only ever reads the cache.

5.2 Selection policy and device arguments

Table 7. Acceleration modes (`RuntimeConfig.acceleration`).

Mode	Devices chosen	Failure behaviour
auto (default)	all discrete/unknown GPUs; else the integrated GPU(s); else none	GPU start failure → one retry with devices = [] (CPU)
gpu	same choice	no usable GPU → start refused; GPU start failure → FAILED
cpu	none	—

An integrated GPU is not combined with discrete ones: splitting layers onto a device that shares system memory would only add transfer overhead. The chosen devices become explicit arguments, so llama.cpp never picks devices on its own:

```
devices == [] -> --device none --n-gpu-layers 0
devices != [] -> --device Vulkan0[,Vulkan1,...] --n-gpu-layers auto
```

With `--n-gpu-layers auto` and llama.cpp's `--fit` enabled by default, llama.cpp measures projected per-device memory use and assigns as many layers as fit (§6.3); the remaining layers run on the CPU backend.

5.3 Start sequence and CPU fallback

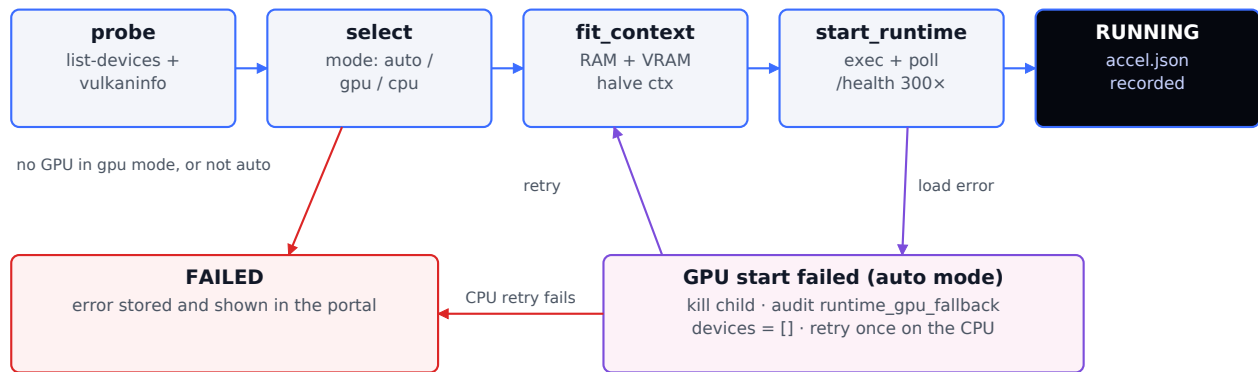


Figure 3. Runtime start in the manager loop. The fallback is attempted once, only in auto mode and only when a GPU was selected; the note "GPU start failed, running on the CPU: <error>" is stored with the runtime and shown in the portal.

`start_runtime()` writes `runtime/<id>.accel.json` (devices, fallback note, log offset), launches the process with its output appended to `runtime/<id>.log` in a new session, and polls GET `http://127.0.0.1:8090/health` up to 300 times with a 2 s client timeout. A process that exits during load raises a `ValueError` carrying llama.cpp's first error line (for example an unknown architecture or `ErrorOutOfDeviceMemory`). In auto mode with devices selected, the child is killed, the event is audited as `runtime_gpu_fallback`, and the start is repeated with `devices = []`, recomputing the context fit without GPU memory.

6. Memory model and context fitting

6.1 KV cache estimate

The key/value cache is estimated from the GGUF header read during discovery (`block_count`, `embedding_length`, `attention.head_count`, `attention.head_count_kv`, `attention.key_length`), assuming the F16 cache types that are llama.cpp's defaults (`cache_type_k` = `cache_type_v` = F16):

```
head_dim = attention.key_length or embedding_length / head_count
kv_bytes = n_ctx · n_layer · 2 (K and V) · head_dim · n_head_kv · 2 (bytes per F16 element)
# without a readable header: kv_bytes ≈ n_ctx · 25 KiB · (file_size / GiB)
```

For hybrid-attention models whose per-layer head counts are arrays, discovery keeps the maximum, so the estimate errs high.

6.2 Compatibility rating

```
required = 0.5 · weights + kv_bytes + projector_bytes + 192 MiB
vram      = Σ (total - 512 MiB) over selected discrete GPUs (integrated GPUs add nothing)
usable    = (ram_total - 1.5 GiB) + vram
resident  = ram_total - 1 GiB
```

Table 8. Classification, evaluated in order (context 4096 in the catalogue).

Condition	Rating
unknown size or not GGUF	INCOMPATIBLE
weights + projector > resident + vram	NOT_RECOMMENDED — weights would be read from disk on every token
required > usable	NOT_RECOMMENDED
required > 0.9 · ram_available + vram	LIMITED
required > 0.6 · usable, or fewer than 4 physical cores	COMPATIBLE
otherwise	OPTIMAL
weights > free model storage (overrides)	INCOMPATIBLE

Counting half of the weights reflects that llama.cpp memory-maps the file: pages not currently needed can be evicted and re-read. The factor was calibrated on small models (a 1.57 GiB model at context 4096 loaded in 18 s on a 3 GiB VM and answered without swap). It is optimistic for models whose weights approach total RAM, since decoding touches every weight of a dense model per token (§14).

6.3 Context fitting at start and interaction with --fit

The default `RuntimeConfig.context` is 101024 tokens, a ceiling rather than a promise. Before each start `fit_context()` computes:

```
extra      = Σ (free - 512 MiB) over selected discrete GPUs
available  = MemAvailable - 128 MiB + extra
capacity   = MemTotal - 1 GiB + extra
if weights > capacity:                refuse ("would read from disk on every token")
ctx = config.context
if context not set explicitly and trained_context: ctx = min(ctx, <arch>.context_length)
while not explicit and ctx > 1024 and required(ctx) > available: ctx = max(1024, ctx // 2)
if required(ctx) > available:          refuse with needed / available MiB
persist ctx in runtime_instances.config; audit runtime_context_reduced
```

The resulting value is always passed as `--ctx-size`. This matters for llama.cpp's own fitting logic (`common/fit.cpp`): `--fit` is on by default with a free-memory target of 1024 MiB per device and a

minimum context of 4096, but it reduces the context only when the context was left unset (`n_ctx == 0`). Because AIOS sets it, llama.cpp's fit only distributes layers: it measures projected memory for the requested parameters, then fills devices back to front with whole layers (for MoE models, dense tensors first with expert tensors in system memory), searching the number of layers per device by the method of false position until a one-layer difference separates a fitting from a non-fitting configuration. The two fits are complementary: AIOS decides the context from whole-system memory, llama.cpp decides placement from per-device budgets.

7. The llama-server invocation

`runtime.arguments()` builds the complete command line. Nothing relevant is left to defaults that could change between llama.cpp revisions.

```
/opt/aios/runtime/bin/llama-server \
--model /var/lib/aios/models/<uuid>.gguf --alias <uuid> --host 127.0.0.1 --port 8090 \
--ctx-size <fitted> --threads <calibrated> --batch-size 512 --parallel 1 --timeout 300 \
--metrics --no-webui \
--device Vulkan0 --n-gpu-layers auto           # or: --device none --n-gpu-layers 0
--mmproj /var/lib/aios/models/<uuid>.mmproj.gguf [--no-mmproj-offload when on the CPU]
--reasoning-budget <n> --reasoning-budget-message "\n\nI have reasoned enough; ..."
```

Table 9. Parameters passed by AIOS, compared with llama.cpp defaults at the pinned commit.

Option	AIOS value	llama.cpp default	Rationale
<code>--ctx-size</code>	fitted (≤ 101024 , $\leq$ trained)	0 = trained context	explicit; context decided from measured memory (§6.3)
<code>--threads</code>	benchmark recommendation, else physical cores	-1 (auto)	logical threads on SMT hosts slow decoding
<code>--batch-size</code>	512 (LOW_MEMORY profile: 128)	2048	bounds compute buffers; ubatch stays at its default 512
<code>--parallel</code>	1 (1-16)	1	slots share --ctx-size; kv_unified stays disabled
<code>--timeout</code>	300 s (10-3600)	3600 s	server read/write timeout; also the gateway slot wait
<code>--device</code>	Vulkan list or none	all devices	selection is made by AIOS (§5.2)
<code>--n-gpu-layers</code>	auto or 0	auto (-1)	auto lets --fit place layers
<code>--fit</code>	not passed (on)	on, target 1024 MiB/device	layer placement only, context is explicit
<code>--mmproj</code>	projector path if installed	none	image input for vision models (§9)
<code>--no-mmproj-offload</code>	only when devices = []	offload enabled	the projector would otherwise pick a GPU on its own
<code>--reasoning-budget</code>	auto: $\max(256, \text{ctx}/\text{parallel}/2)$	-1 (unlimited)	thinking models always keep room to answer (§8)
<code>--metrics</code>	enabled	disabled	Prometheus metrics, re-exported by the control plane
<code>--no-webui</code>	set	UI enabled	the built-in UI is not exposed
<code>--flash-attn, cache types</code>	not passed	auto, F16/F16	engine chooses per backend; KV estimate assumes F16
<code>--numa, --mlock, --embedding</code>	only when configured	off	--embedding --pooling mean for embedding runtimes
<code>taskset -c</code>	only with CPU affinity	—	validated against sched_getaffinity

The runtime profile adjusts the defaults: AUTO uses the calibrated thread count, LOW_MEMORY caps context at 2048 and batch at 128, MAX_PERFORMANCE uses all physical cores. Explicit HugeTLB is refused: llama.cpp maps the model file, so only transparent huge pages can apply. The environment of the process is the runtime manager's:

LD_LIBRARY_PATH=/opt/aio/runtime/lib, working directory /opt/aio/app, supplementary groups render and video.

8. Reasoning budget

Reasoning models emit a thinking segment before the answer. Small quantised models can think until the context is full and stop with no answer. llama.cpp implements a per-request budget in the sampler (reasoning_budget_tokens): when the budget is exhausted it injects reasoning_budget_message followed by the model's end-of-thinking tag, forcing the transition to the answer.

Table 10. RuntimeConfig.reasoning_budget.

Value	Passed to llama-server	Meaning
-2 (default)	max(256, ctx_size / parallel / 2)	half of what one request may use goes to thinking
-1	-1	unlimited
0	0	thinking ends immediately
N > 0	N	fixed budget

Clients can still disable thinking per request with "chat_template_kwargs": {"enable_thinking": false}; Open WebUI's automatic tasks are configured that way (§10.3).

9. Multimodal input

Vision-language models in GGUF form are two files: the language model and a multimodal projector (mmproj, GGUF architecture clip or general.type = mmproj) holding the vision encoder and the projection into the embedding space. llama-server loads the projector with --mmproj; without it every image is rejected with "image input is not supported".

Discovery and pairing

Projector files are recognised by name ((^[^_\.])mmproj([_\.]|\$), not a shard, ≤ 64 GiB) and never listed as models. Within one repository (or one release, or one manifest model), each model file receives the best projector by this ranking: a projector whose name tokens, after removing mmproj and the precision, contain a token absent from the model file name belongs to another model and is excluded; among the rest, more matching name tokens rank first, then precision in the order F16, BF16, Q8_0, F32, then smaller size.

```
unsloth/Qwen3-VL-30B-A3B-Thinking-1M-GGUF
  Qwen3-VL-30B-A3B-Thinking-1M-UD-TQ1_0.gguf -> mmproj-F16.gguf  (BF16, F32 also published)
bartowski-style per-size projectors
  google_gemma-3-4b-it-Q4_K_M.gguf             -> mmproj-google_gemma-3-4b-it-f16.gguf
  google_gemma-3-12b-it-Q4_K_M.gguf            -> mmproj-google_gemma-3-12b-it-f16.gguf
```

Download, verification and start

The download job fetches the model to <job>.part and the projector to <job>.mmproj.part, both resumable, with progress accounted over the combined size. The model is verified (SHA256 when upstream publishes one, structural GGUF validation, draft detection); the projector is verified with its own SHA256 and must parse as a projector GGUF. A projector that fails verification is discarded while the verified model is still installed, and the job records "Installed without image input: <reason>". Models installed before projectors were tracked can fetch only the projector: the lookup also searches newer revisions of the same upstream file. At start the projector adds its full size to the memory estimate, since it is read into memory whole.

10. Request path

10.1 NGINX

```
ssl_protocols TLSv1.2 TLSv1.3; client_max_body_size 32m;
proxy_http_version 1.1; proxy_read_timeout 3600s; proxy_send_timeout 3600s;
location /v1/ {
    proxy_buffering off;                                # server-sent events pass through unbuffered
    proxy_pass http://127.0.0.1:8081;
}
location /api/v1/aios/auth/ { limit_req zone=login burst=10 nodelay; proxy_pass http://127.0.0.1:8081; }
location = /metrics { allow 127.0.0.1; allow ::1; deny all; proxy_pass http://127.0.0.1:8081; }
location / {
    # Open WebUI behind the AIOS session
    auth_request /_aios/chat-session;
    auth_request_set $chat_email $upstream_http_x_aios_email;
    proxy_set_header X-AIOS-Email $chat_email;          # also name and role
    proxy_pass http://127.0.0.1:8080;
}
```

proxy_buffering off on /v1/ lets server-sent events reach the client token by token. Open WebUI has no credential store of its own: NGINX validates the AIOS session with a subrequest and injects trusted identity headers that the client cannot set.

10.2 Gateway semantics

POST /v1/{chat/completions, completions, embeddings} requires Authorization: Bearer <key>, compared in constant time with /etc/aios/secrets/inference-key. The model field is the installation UUID (or the configured default model). A published model that is not running is loaded on request: under a global load lock, any other model is asked to stop unless it is serving a request, the target's desired state is set to RUNNING, and the gateway waits up to 900 s for RUNNING or FAILED. Requests are then admitted through a per-model semaphore of parallel permits with a wait bounded by the runtime timeout, and proxied with the body re-serialised and model rewritten. Streaming responses are forwarded line by line while usage counters are extracted; non-streaming bodies are capped at 32 MiB.

Table 11. Gateway status codes.

Status	Condition
401	missing or wrong inference key
404	model not installed and published, or unknown endpoint
409	file is not a servable model, or another model is answering requests
429	no inference slot freed within the runtime timeout
502	runtime unreachable or response larger than 32 MiB
503	model could not be loaded (reason from llama.cpp) or load exceeded 900 s

10.3 Open WebUI configuration

Open WebUI runs offline (OFFLINE_MODE=true, HF_HUB_OFFLINE=1, telemetry and update checks off, code execution off) with the gateway as its only OpenAI connection. Because every automatic task runs on the same local model, AIOS sets defaults that keep the model free between answers: title generation stays but runs with TASK_MODEL_PARAMS = {"max_tokens": 128, "reasoning_budget_tokens": 0, "chat_template_kwargs": {"enable_thinking": false}}; tag, follow-up, autocomplete and query-generation tasks are disabled; built-in tools are disabled by default model metadata, since their schemas add about 6000 tokens to each request.

11. Model ingestion and validation

Each provider discovers differently. Hugging Face is queried per author; ModelScope has no such endpoint, so its search index is queried per publisher in two orders — newest first and by relevance — and only repositories that publisher owns are kept, with the limit spent one publisher at a time. A provider that needs configuration, such as GitHub releases, reports NOT CONFIGURED rather than an error.

Discovery is metadata-only. For Hugging Face, GET /api/models/<id>?blobs=true returns file sizes and LFS SHA256 digests. Files differing only in quantisation share one tensor layout, so one header per layout is read with an HTTP range request of 256 KiB, widened to 2 MiB when the dimensions sit after a long chat template (at most 16 header reads per model). The scan stops before the vocabulary once the architecture and the five sizing keys are known. Drafts for speculative decoding (<arch>.target_layers, or fewer than two tensors per declared block), imatrix files, adapters, split archives and architectures absent from the bundled llama.cpp are excluded.

Table 12. Structural GGUF validation limits applied to downloaded files.

Check	Limit
magic / version	"GGUF", version 2 or 3
tensor count	1 ... 100 000
metadata entries	$\leq 10\,000$, unique keys ≤ 512 bytes
metadata budget	≤ 64 MiB read
string length	≤ 1 MiB
array length	$\leq 4\,000\,000$, no nested arrays
tensor dimensions	1-4 dims, each $1 \dots 2^{32}$, type id ≤ 100
alignment	power of two ≤ 4096 ; tensor offsets aligned and inside the file

Downloads stream in 256 KiB chunks with HTTP range resume, pinned resolved addresses (the connection goes to the validated IP with SNI and Host preserved), credentials stripped on cross-host redirects and fsync before verification. Transient HTTP or database errors keep the partial file and retry after $2^n \cdot 5$ s, up to five attempts. Verified files are made read-only (0440) and renamed into models/<uuid>.gguf.

12. Isolation of the inference services

All long-running AIOS services run as the unprivileged aios user with an empty capability bounding set and systemd sandboxing. The runtime manager unit illustrates the profile:

```
[Service]
User=aios Group=aios SupplementaryGroups=systemd-journal render video
Environment=LD_LIBRARY_PATH=/opt/aios/runtime/lib LimitMEMLOCK=infinity UMask=0027
NoNewPrivileges=yes ProtectSystem=strict ProtectHome=yes PrivateTmp=yes
ReadWritePaths=/var/lib/aios /etc/aios /var/log/aios
ProtectKernelTunables=yes ProtectKernelModules=yes ProtectControlGroups=yes
RestrictSUIDSGID=yes LockPersonality=yes CapabilityBoundingSet=
RestrictAddressFamilies=AF_UNIX AF_INET AF_INET6 AF_NETLINK
[Unit]
After=aios-firstboot.service network.target aios-nvidia-driver.service
```

PrivateDevices is deliberately not set on the two units that open GPUs, since it would hide /dev/dri and /dev/nvidia*. llama-server listens on loopback only, with no API key of its own; the gateway is the only path to it from the network.

13. Verification and measurements

Automated tests (283 at the time of writing) run against a real SQLite database, the real GGUF parser and controlled HTTP transports; the GPU tests feed recorded `--list-devices` and `vulkaninfo` output and a fake PCI tree to the NVIDIA flavour decision. End-to-end checks run on a lab appliance installed from the release ISO: a KVM guest with 2 vCPUs of an Intel Core i5-7600T (AVX2) and 3–4 GiB of RAM, without a GPU.

Table 13. Lab measurements (single request, one slot). The software-renderer row validates the GPU code path, not GPU performance.

Scenario	Model	Start	Prompt eval	Generation
CPU, <code>--device none</code>	Qwen3.5-0.8B Q4_K_M, ctx fitted to 25256	9–12 s	45 tok/s (16 tok)	25.4 tok/s (60 tok)
Vulkan0 = lavapipe (forced)	same	36 s	0.36 tok/s	1.69 tok/s
CPU + projector	Qwen3-VL-2B-Instruct Q4_K_M + mmproj-F16, ctx 6314 (4 GiB VM)	18 s	—	image question answered in 1.5 s ("Red and blue")

The lavapipe run exercises detection, selection, the Vulkan backend, device buffers and the portal end to end; its speed is irrelevant because the "GPU" is the same CPU emulated through LLVM. Performance on physical GPUs depends on the card, driver and quantisation and is not benchmarked by the project.

14. Limitations and future work

- **Large models near total RAM.** The compatibility factor of 0.5-weights (\$6.2) is calibrated on small models. For a dense model whose weights approach total memory, decoding touches every page per token, so the fitted context can leave the weights partly evicted and turn each token into disk reads. The estimate should count the whole file when it is large relative to RAM.
- **Integrated GPUs and UMA copies.** Layers offloaded to an integrated GPU are copied into Vulkan buffers carved from system RAM (\$4.4) while the memory-mapped file remains in the page cache. AIOS does not yet add that duplication to its estimate, and on UMA systems generation speed is bounded by the same memory bandwidth as the CPU path; the gain is mostly in prompt processing.
- **Offload reporting.** At this llama.cpp revision the default log verbosity no longer prints offloaded N/M layers to GPU, so the portal shows the devices used but not the layer split.
- **Vulkan versus vendor runtimes.** CUDA can outperform Vulkan on NVIDIA GPUs for some kernels; the image trades that for uniform support across vendors.
- **One model at a time.** Loading a second model unloads the first unless it is serving a request.
- **New kernels.** Bundled NVIDIA module packages match the image's kernel; after a kernel update the modules are fetched from the Ubuntu archive at the next boot, which needs network access.

15. The image engine

From version 1.5 the appliance carries a second inference engine, `stable-diffusion.cpp`, built from a pinned commit with its own pinned `ggml` fork. It is compiled exactly like the language engine — `GGML_NATIVE=OFF`, `GGML_BACKEND_DL=ON`, `GGML_CPU_ALL_VARIANTS=ON`, `SD_VULKAN=ON` — but installed under `/opt/aios/imaging`, with its backends symlinked next to its executable and `LD_LIBRARY_PATH` set per process, so neither engine can load the other's `ggml`. The image build asserts that this engine, too, lists a Vulkan device before the ISO is written.

Table 14. Differences between the two runtimes.

	Language models	Image models
Engine / port	llama-server, 127.0.0.1:8090	sd-server, 127.0.0.1:8091
Model files	one GGUF, plus a projector for vision	one checkpoint, or a transformer with text encoders and a VAE (up to four files, often from three repositories)
Memory	weights are memory-mapped and partly evictable; KV cache scales with context	every file is resident for the whole denoising pass, plus ~1.5 GiB of buffers; no context
Device choice	<code>--device Vulkan0 --n-gpu-layers auto</code>	<code>--backend Vulkan0</code> , with <code>--offload-to-cpu</code> when the card has less free memory than the model
Request path	<code>/v1/chat/completions</code> , streamed	<code>/v1/images/generations</code> , one picture per request, up to 30 minutes
Concurrency	one language model loaded	one image model loaded, beside a language model

Generation parameters that the OpenAI-compatible route does not carry — steps, guidance scale, sampler — are passed as process defaults per model family (`--steps`, `--cfg-scale`, `--sampling-method`, `--width`, `--height`); a single request overrides them through the engine's own extension block embedded in the prompt. Measured on four threads of an Intel Core i5-7600T without a GPU, Stable Diffusion 1.5 produces a 512×512 picture in 20 steps in 218 s, and a 256×256 picture in 6 steps in 44 s; a discrete GPU reduces this to seconds.

References

- [1] ggml-org, “llama.cpp”, commit 5266f24da75dc449bd56cbcd7addb9c8e4a6a73e. <https://github.com/ggml-org/llama.cpp>
- [2] ggml-org, “GGUF file format”. <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>
- [3] Khronos Group, “Vulkan 1.3 Specification”, including `VK_EXT_memory_budget` and `VK_KHR_cooperative_matrix`. <https://registry.khronos.org/vulkan/>
- [4] Mesa 3D, “Vulkan drivers: ANV, hasvk, RADV, NVK, lavapipe”. <https://docs.mesa3d.org/>
- [5] NVIDIA, “open-gpu-kernel-modules” and supported GPU lists for the 580 driver branch. <https://github.com/NVIDIA/open-gpu-kernel-modules>
- [6] Canonical, “Ubuntu 24.04 LTS hardware enablement stack”. <https://ubuntu.com/kernel/lifecycle>
- [7] Open WebUI 0.11.3 documentation, environment configuration. <https://docs.openwebui.com/>
- [8] leejet, “stable-diffusion.cpp”, commit cc515a01f9d0e3f6b975234cc934b807f55bcd35. <https://github.com/leejet/stable-diffusion.cpp>
- [9] AIOS project documentation: architecture, GPU acceleration, models and runtimes, image generation, verification (docs/ in the source repository).